

Критерии оценки языков программирования

Зачем оценивать языки

- Выбор языка для проекта: прототип на Python/JavaScript, промышленные системы на Java/Rust/C++.
- Выбор языка для обучения: важны простота, наглядность, хороший инструментарий (Pascal, Python и др.).
- Оценка стоимости сопровождения: насколько просто читать и менять код через несколько лет.

Для кого важна оценка

- разработчик (что учить и на чем писать свои проекты);
- команда (на чем реализовать новый продукт);
- индустрия (почему «старые» языки живут десятилетиями).

Критерии

- Читабельность - как быстро другой разработчик поймёт, что делает программа.
- Легкость создания - насколько просто писать и изменять код на языке.
- Надежность - насколько язык помогает избегать и обнаруживать ошибки.
- Стоимость - обучение, инструменты, производительность, поддержка кода в долгую.

Характеристики

- Простота\Ортогональность
- Управляющие структуры
- Типы и структуры данных
- Синтаксическая структура
- Поддержка абстракции
- Выразительность
- Проверка типов
- Обработка исключительных ситуаций
- Ограниченное совмещение имен

Простота\Ортогональность

Ортогональность — это когда небольшое число базовых конструкций можно комбинировать по единым правилам, и не возникает неожиданных ограничений.

Влияет на читабельность, легкость создания и надежность

- Количества конструкций языка
- Множественность свойств, способов выполнить действие

`x = 2 * (if a > b then a else b end)` — выражение `if` можно использовать как часть другой конструкции.

Пример хорошей ортогональности

```
def apply_twice(f, x):  
    return f(f(x))  
  
def inc(x):  
    return x + 1  
  
print(apply_twice(inc, 10))  # 12
```

Пример плохой ортогональности

- массивы нельзя возвращать и присваивать так же свободно, как простые типы;
- строки — это особым образом оформленные массивы `char`, к которым нельзя применять все операции с массивами.

Управляющие структуры

- Наличие оператора goto
- Ограничения на оператор goto
- Наличие специализированных управляющих структур: while, for, switch

Влияет на читабельность, легкость создания и надежность

Пример

```
// Goto (Плохо)
if (x <= 0) goto Label1;
y = x;
goto Label2;
Label1: y = -x;
Label2: print(y);
```

```
// Structured (Хорошо)
if (x > 0) y = x;
else y = -x;
print(y);
```

Пример использования специализированной конструкции (switch / match):

```
switch (day) {  
    case 1: printf("Monday"); break;  
    case 2: printf("Tuesday"); break;  
    // ...  
    default: printf("Unknown");  
}
```

Такие конструкции уменьшают количество вложенных `if`, делают код более читабельным.

Пример цикла for-each (Go и Rust):

```
sum := 0
for _, x := range nums {
    sum += x
}
```

```
let sum: i32 = nums.iter().sum();
```

Оба примера скрывают детали обхода коллекции и уменьшают количество ошибок с индексами.

Типы и структуры данных

- адекватное определение типов и структур данных

В старом Fortran или C: `int flag = 1;` (непонятно, это число или истина/ложь).

В современных языках: `bool isReady = true;` (читабельно и надежно).

Сильная и слабая типизация:

- слабая типизация: язык неявно приводит типы (может скрыть ошибку);
- сильная типизация: большинство опасных приведений

Пример выразительных типов

```
enum Day { MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY }  
  
Day meetingDay = Day.MONDAY; // вместо int day = 1;
```

```
type Day = "Mon" | "Tue" | "Wed" | "Thu" | "Fri" | "Sat" | "Sun";  
  
let meetingDay: Day = "Mon"; // тип не позволит записать "Monday"
```

Синтаксическая структура

- Формы идентификаторов
- Специальные слова
- Форма и значение. Когда одна и та же конструкция означает разные вещи в зависимости от контекста.

Пример

Ключевое слово `static` в C/C++:

- Внутри функции: сохраняет значение между вызовами.
- Глобально: ограничивает видимость переменной текущим файлом.
- В классе (C++): переменная общая для всех объектов.

Также синтаксис влияет на количество «шума»:

```
// Java  
int sum = 0;  
for (int x : list) {  
    sum += x;  
}
```

```
# Python  
sum = 0  
for x in list:  
    sum += x
```

![[C++ vs Python.mp4]]

Поддержка абстракции

- абстракция процессов
- абстракция данных

Пример:

- Абстракция процесса: Вызов `sort(array)`. Нам не важно, Bubble Sort там или Quick Sort, мы просто сортируем.
- Абстракция данных: Тип Стек или Дерево. Мы используем методы `push/pop`, не думая о массивах и указателях внутри.

Пример до и после абстракции:

До: дублирование кода

```
total = 0
```

```
for x in xs:
```

```
    total += x
```

```
avg = total / len(xs)
```

После: абстракция процесса

```
def average(xs):
```

```
    return sum(xs) / len(xs)
```

```
avg = average(xs)
```

// Абстракция данных в Rust через новый тип

```
struct Stack<T> {  
    items: Vec<T>,  
}  
  
impl<T> Stack<T> {  
    fn push(&mut self, item: T) { self.items.push(item); }  
    fn pop(&mut self) -> Option<T> { self.items.pop() }  
}
```

Выразительность

- более удобные и менее громоздкие конструкции

Одна и та же задача разными способами (подсчёт квадратов положительных чисел):

```
result = []  
for x in xs:  
    if x > 0:  
        result.append(x * x)
```

```
# Более выразительно  
result = [x * x for x in xs if x > 0]
```

Проверка типов

- Динамическое приведение
- Статические приведение

JS: "1" + 2 дает "12". Ошибка может всплыть только при запуске (плохая надежность).

Java/C#: Компилятор выдаст ошибку на этапе сборки (хорошая надежность).

Где ловится ошибка

- динамическая проверка типов — ошибка проявится только при выполнении проблемной ветки;
- статическая проверка типов — многие ошибки обнаруживаются до запуска программы.

Обработка исключительных ситуаций

- Неструктурная обработка исключений (обработчик регистрируется)
- Структурная обработка исключений (try-throw-catch)
- Блоки с гарантированным завершением

Влияет на легкость создания и надежность

Коды ошибок vs исключения (1)

```
int read_file(const char* path) {  
    FILE* f = fopen(path, "r");  
    if (!f) return -1; // код ошибки  
    // ...  
    fclose(f);  
    return 0;  
}
```

Коды ошибок vs исключения (2)

```
def read_file(path):  
    try:  
        with open(path, "r") as f:  
            data = f.read()  
            return data  
    except OSError as e:  
        print("Ошибка чтения файла:", e)
```

Ограниченное совмещение имен

- Подпрограммы
- Переменные
- Константы

Опасность совпадения имён (shadowing):

```
count = 10  # глобальная переменная

def process(items):
    count = 0  # перекрывает глобальную
    for item in items:
        count += 1
    return count
```

Разные области видимости могут использовать одно и то же имя, что затрудняет понимание программы.

```
let count = 10; // внешняя переменная

fn process(items: &[i32]) -> usize {
    let mut count = 0; // скрывает внешнюю
```

Сравнение языков по критериям (1)

Читабельность:

- Kotlin, Swift, TypeScript — современный, достаточно лаконичный синтаксис; много «говорящих» конструкций.
- Python — максимально простой и читаемый синтаксис, но иногда не хватает явных типов.
- Go — намеренно простой, но более «низкоуровневый» по стилю.
- Rust — выразителен, но синтаксис сложнее для новичков (ownership, lifetimes).

Сравнение языков по критериям (2)

Лёгкость создания:

- Python, Kotlin, TypeScript, Swift — высокая скорость разработки, развитые IDE.
- Go — простота языка + быстрые компиляции.
- Rust — самая высокая «входная цена», но компилятор помогает писать безопасный код.

Сравнение языков по критериям (3)

Надёжность:

- Rust, Kotlin, Swift, TypeScript, Go — строгая статическая типизация, много ошибок ловится на этапе компиляции.
- Python — надёжность зависит от тестов и дисциплины; типы можно усилить аннотациями и mypy.

Сравнение языков по критериям (4)

Стоимость:

- Python, TypeScript, Go — низкий порог входа и широкое сообщество/библиотеки.
- Kotlin, Swift — завязаны на определённые экосистемы (JVM, Apple), но хорошо поддерживаются.
- Rust — требует большего обучения, но окупается там, где критичны производительность и безопасность.

Дополнительное задание

- Проанализировать языки C++, Kotlin, Python, Go, Rust, Swift, TypeScript
- Найти и описать 5 кейсов, когда в одном языке какой-то элемент языка плохо влияет на определенную характеристику, а в другом языке(ах) этот же элемент сделан по другому и положительно влияет на эту же характеристику.